Лабораторна робота №7.
Використання ШІ-асистентів у розробці

Мета роботи: Ознайомитися з використанням ШІ-асистентів під час розробки програмного забезпечення на прикладі простого веб-сервісу на Python із використанням Flask. У процесі виконання роботи студент навчиться: використовувати ШІ для пояснення програмного коду; формулювати прості та уточнені промпти; використовувати ШІ для доповнення програмного коду; знаходити та виправляти помилки за допомогою ШІ; використовувати ШІ для створення тестів; перевіряти результат роботи ШІ; ознайомиться з поняттям ШІ-агента..

Виконати цей код на своїх комп'ютерах.

1. Файл `app.py`

```
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

# Список для зберігання завдань
tasks = []

# Маршрут для головної сторінки
@app.route('/')
def home():
    return "Вітаю у веб-сервісі на Flask!"

# Маршрут для перегляду списку завдань
@app.route('/tasks')
def task_list():
    return render_template('tasks.html', tasks=tasks)

# Маршрут для додавання нового завдання
@app.route('/add', methods=['GET', 'POST'])
def add_task():
    if request.method == 'POST':
        new_task = request.form['task']
        if new_task:  # Перевірка, чи поле не порожнє
            tasks.append(new_task)
        else:
            return "Завдання не може бути порожнім", 400
        return redirect(url_for('task_list'))
    return render_template('add_task.html')

# Запуск сервера
if __name__ == '__main__':
    app.run(debug=True)
```

2. Файл `templates/tasks.html`

```
<!doctype html>
<html lang="en">
```

```
<head>
    <meta charset="UTF-8">
    <title>Список завдань</title>
</head>
<body>
    <h1>Список завдань</h1>
    <ul>
        {% for task in tasks %}
            <li>{{ task }}</li>
        {% endfor %}
    </ul>
    <a href="{{ url_for('add_task') }}">Додати нове завдання</a>
</body>
</html>
```

3. Файл `templates/add_task.html`

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Додати завдання</title>
</head>
<body>
    <h1>Додати нове завдання</h1>
    <form method="POST">
        <input type="text" name="task" placeholder="Назва завдання">
        <button type="submit">Додати</button>
    </form>
    <a href="{{ url_for('task_list') }}">Назад до списку завдань</a>
</body>
</html>
```

Запуск проекту

1. Створіть структуру директорій для вашого проекту:

   ```
   my_flask_app/
   ├── app.py
   └── templates/
       ├── add_task.html
       └── tasks.html
   ```

2. Встановіть Flask, якщо він ще не встановлений:

   ```
   pip install flask
   ```

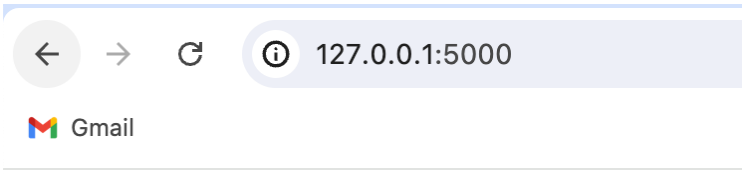
```
(base) Roman_Mysiuk@EPUALVIW0D67 Documents % cd folder\ 2
(base) Roman_Mysiuk@EPUALVIW0D67 folder 2 % ls
app.py          templates
(base) Roman_Mysiuk@EPUALVIW0D67 folder 2 % python app.py
Traceback (most recent call last):
  File "/Users/Roman_Mysiuk/Documents/folder 2/app.py", line 1, in <module>
    from flask import Flask, render_template, request, redirect, url_for
ModuleNotFoundError: No module named 'flask'
(base) Roman_Mysiuk@EPUALVIW0D67 folder 2 % pip install flask
Collecting flask
```

3. Запустіть сервер:

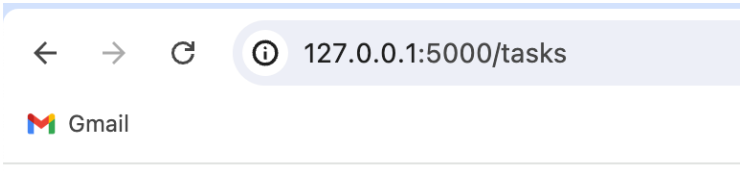
```
python app.py
```

4. Відвідайте сторінки `http://127.0.0.1:5000/` для головної сторінки, `http://127.0.0.1:5000/tasks` для перегляду завдань та `http://127.0.0.1:5000/add` для додавання нових завдань.

```
(base) Roman_Mysiuk@EPUALVIW0D67 folder 2 % python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 462-483-293
127.0.0.1 - - [31/Aug/2024 13:16:49] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [31/Aug/2024 13:16:50] "GET /favicon.ico HTTP/1.1" 404 -
```



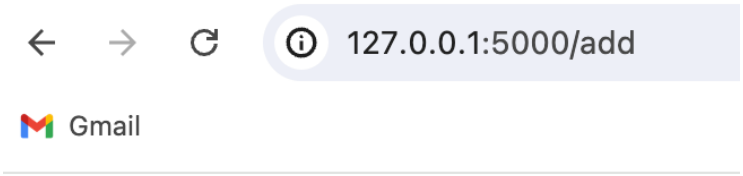
Вітаю у веб-сервісі на Flask!



Додати нове завдання

Назва завдання Додати

Назад до списку завдань



Список завдань

Додати нове завдання

1. **Створення проекту Flask:**
   - **Крок 1:** Встановіть Flask за допомогою pip:

     ```
     pip install flask
     ```

   - **Крок 2:** Створіть основний файл для вашого проекту, наприклад, `app.py`:

     ```
     from flask import Flask

     app = Flask(__name__)

     @app.route('/')
     def home():
         return "Вітаю у веб-сервісі на Flask!"

     if __name__ == '__main__':
         app.run(debug=True)
     ```

   - **Крок 3:** Запустіть сервер командою:

     ```
     bash
     Copy code
     python app.py
     ```

   - Це відкриє ваш веб-сервіс на локальному сервері, зазвичай за адресою `http://127.0.0.1:5000`.
2. **Реалізація базових маршрутів:**
   - **Крок 4:** Додайте маршрути для різних сторінок:

     ```
     from flask import render_template

     @app.route('/tasks')
     def tasks():
         tasks = ["Завдання 1", "Завдання 2", "Завдання 3"]
         return render_template('tasks.html', tasks=tasks)
     ```

   - **Крок 5:** Створіть папку `templates` і додайте HTML-файл `tasks.html`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Список завдань</title>
</head>
<body>
    <h1>Список завдань</h1>
    <ul>
        {% for task in tasks %}
            <li>{{ task }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

3. **Форма для введення даних:**
    - **Крок 6:** Додайте маршрут та форму для додавання нових записів:

```
from flask import request, redirect, url_for

tasks = []

@app.route('/add', methods=['GET', 'POST'])
def add_task():
    if request.method == 'POST':
        new_task = request.form['task']
        tasks.append(new_task)
        return redirect(url_for('tasks'))
    return render_template('add_task.html')
```

    - **Крок 7:** Створіть HTML-файл `add_task.html`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Додати завдання</title>
</head>
<body>
    <h1>Додати нове завдання</h1>
    <form method="POST">
        <input type="text" name="task" placeholder="Назва
завдання">
        <button type="submit">Додати</button>
    </form>
</body>
</html>
```

4. **Збереження та обробка даних:**
    - **Крок 8:** Зберігайте додані завдання в списку `tasks` та відображайте їх на сторінці `/tasks`.

```
@app.route('/tasks')
```

```
def tasks():
    return render_template('tasks.html', tasks=tasks)
```

5. **Валідація даних:**
    - **Крок 9:** Додайте просту валідацію для перевірки, чи користувач ввів текст у форму:

```
if request.method == 'POST':
    new_task = request.form['task']
    if new_task:  # Перевірка, чи поле не порожнє
        tasks.append(new_task)
    else:
        return "Завдання не може бути порожнім", 400
    return redirect(url_for('tasks'))
```

6. **Запуск та тестування:**
    - **Крок 10:** Запустіть сервер та перевірте всі маршрути, додавши завдання через форму, переглядаючи їх у списку та перевіряючи обробку помилок.
7. **Пояснення коду за допомогою ШІ**
    - **Крок 11:** Відкрийте будь-який доступний ШІ-асистент (наприклад, ChatGPT, Gemini, Claude або інший) та надайте йому фрагмент файлу app.py.
    - Використайте такий промпт чи покращений: «**Я вивчаю основи програмування та Flask. Поясни простими словами наведений фрагмент коду. Поясни, що робить кожен рядок код та для чого він потрібен». Додайте для контекту файли app.py, templates/tasks.html, templates/add_task.html.**
    - Додайте нову функцію у запропонований код з допомогою ШІ.
8. **Підготувати звіт.**